

ROB 521H Mobile Robotics and Perception

Deliverables to Simulation

- (1) The video of a successful run of the trajectory rollout on the generated path is attached.

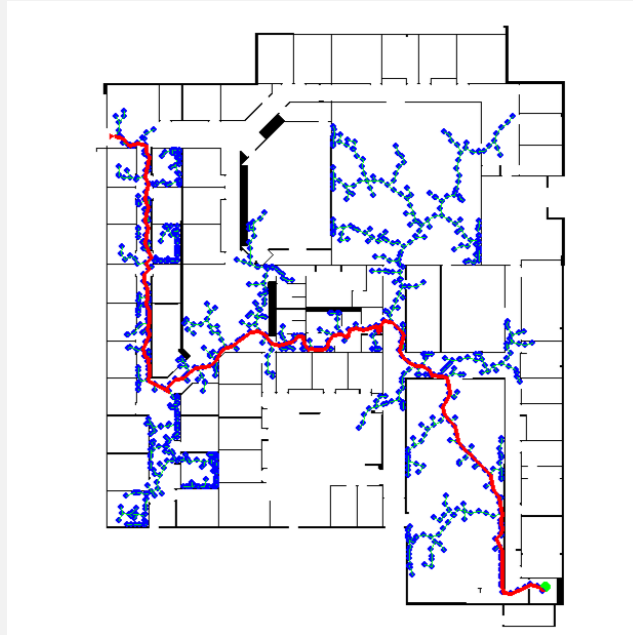


Figure 1: Planning Output of RRT in willowgarageworld_05res.png

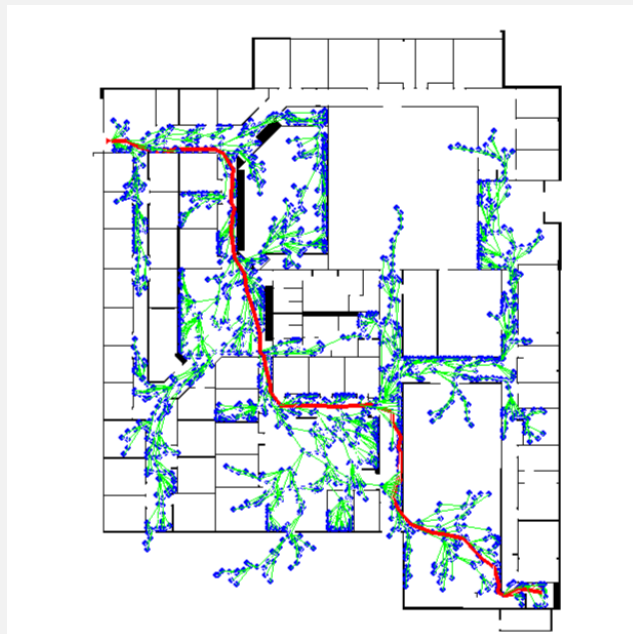


Figure 2: Planning Output of RRT* in willowgarageworld_05res.png

(2) A brief explanation of our code is as follows:

1. For the `rrt_planning()` function, we used a `while` loop to iteratively sample and update the trajectory until a path is found. First, we sample a point from `sample_map_space()` by a probabilistically hybrid method of directly sampling (near) the goal point and sampling randomly within the maze bounds. Then, the nearest point to the sampled point in the planning storage is queried by running `closest_nodes()` leveraging `cKDTree` class in `scipy`. After that, we run `simulate_trajectory():` in `robot_controller()` (which is the only sub-function in this function), we first generate trajectory rollouts^a from a preset series of linear and rotational velocities running `trajectory_rollout()` (while performing collision check and corresponding trajectory truncation for those with collision by `collision_check()`), then select the best trajectory and corresponding combination of linear and rotational velocities with least L2 error from final point to the sample point. Finally, the last pose of the best trajectory is added to the planning storage by creating a new `Node` and connecting to the `cKDTree` if `check_if_duplicate()` judges that it is not within a range of any existing node. Looping stops when reached within a tolerance from the goal point, and the final path is recovered by `recover_path()` then saved.
2. For the `rrt_star_planning()` function, it inherits all procedures of `rrt_planning()` before connecting a new node to the tree. When connecting a new node to the tree: First, a cost^b to come to the node from the nearest existing node is computed by `cost_to_come()` during instantiation of a new `Node`. Then, all existing nodes within a ball radius (`ball_radius()`) are sequentially queried to see if any queried node can be a parent node of the new node while reducing the cost by running `connect_node_to_point()` (while performing collision check and ignoring potential parents if they have collisions in the path to the new node through `collision_check()`), and if we find a parent with a valid lower cost path, rewire the new node to have that node as parent and set its cost as cost to come of parent plus trajectory cost. Then, we continue to check if any node in the queried nodes above can be accessed through the new node with a lower cost, and if any can indeed, also do the rewiring by changing the nearby node's parent to the new node and updating its children's costs recursively by `update_children()`. Finally, if any solution is found (new node near goal node), it will be recovered then saved; if the maximum iteration number is not reached, it will continue to search for solution with lower cost, and if any found, update the saved path.

^aThis is based on simplified differential kinematics of the robot.

^bThis is defined as $\sum_{t=1}^T (x_t - x_{t-1})^2 + (y_t - y_{t-1})^2 + \alpha \cdot \sum_{t=1}^T R_{Robot} \Delta\theta$, where $\Delta\theta \triangleq \min(\frac{\pi}{2} - |\theta_t - \theta_{t-1}|, |\theta_t - \theta_{t-1}|)$.

Deliverables to Real Environment Deployment

- (1) The video of a successful run of the trajectory rollout on the new `Myhal` map is attached.
- (2) The video of a successful open-loop demo of RRT* on the `Myhal` map in real-world is attached.